CS & IT ENGINEERING

Algorithms

Heap Algorithms

Lecture No.- 02

By- Aditya sir

Recap of Previous Lecture

Topic

PYQs on Graph

Topic

Intro to Heaps

Topics to be Covered

Topic

Topic

Topic

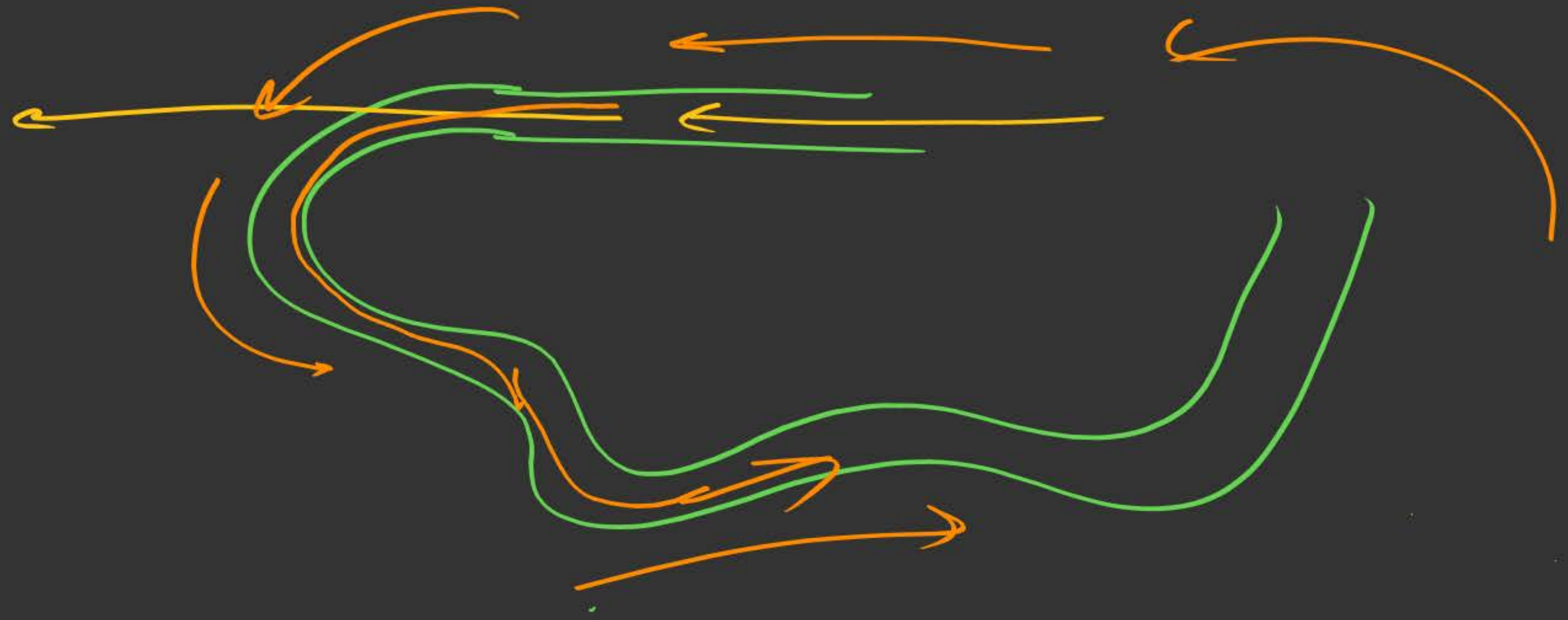
Heap Algos

About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.

@ADITYASIR_PW

Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW

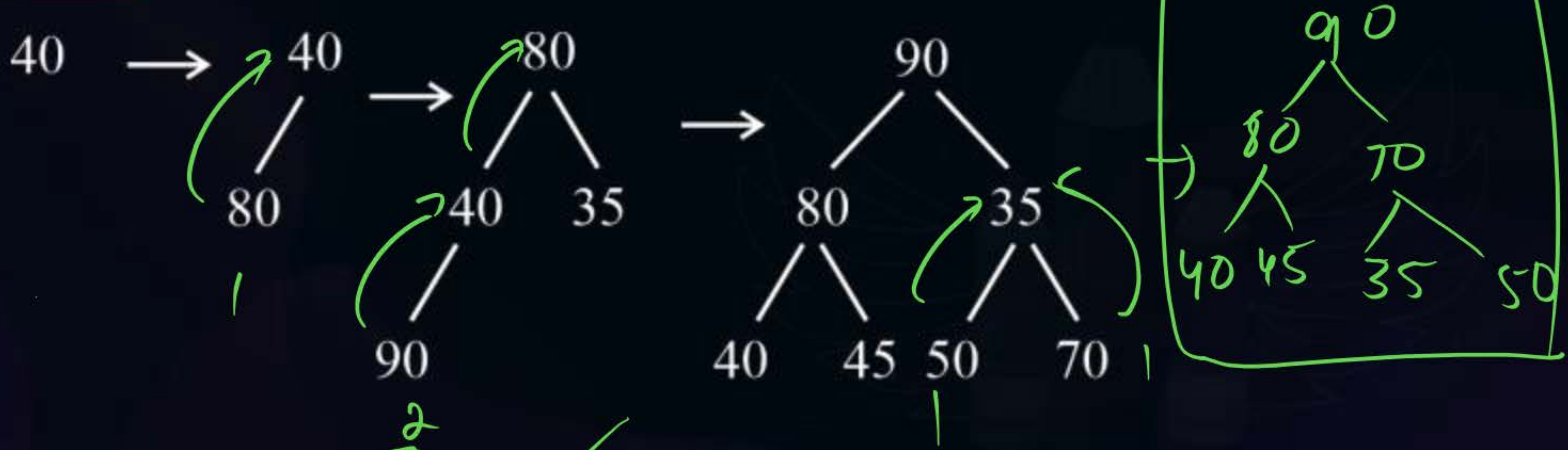


Topic : Heap Algorithms

Test: Given A : [40, 80, 35, 90, 45, 50, 70]

#Q. Create a max-Heap using insertion method.

How many swaps happened during the process?



Total swaps = 1 + 2 + 1 + 1 = 5

Heapify / Build Heap

Topic : Heap Algorithms

Heap Creation using Heapify/Build Heap Method:

Steps:

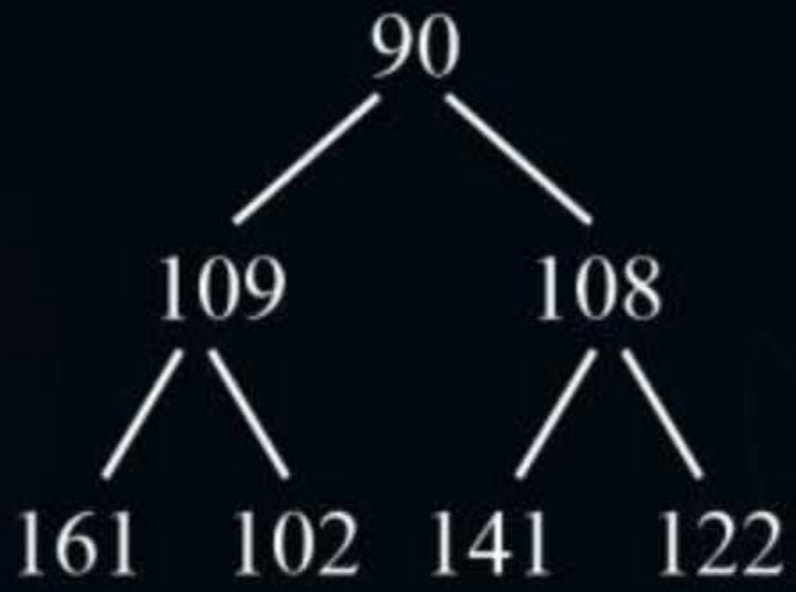
1) Complete Binary Tree already exists of n elements.
2) Make level-by-level adjustments to each node to get to the final Heap.
3) Adjust (Top-Down) a node to move it to ~~tis~~ correct position.

its

Topic : Heap Algorithms

Example: A : [90, 109, 108, 161, 102, 141, 122]

Given CBT is:



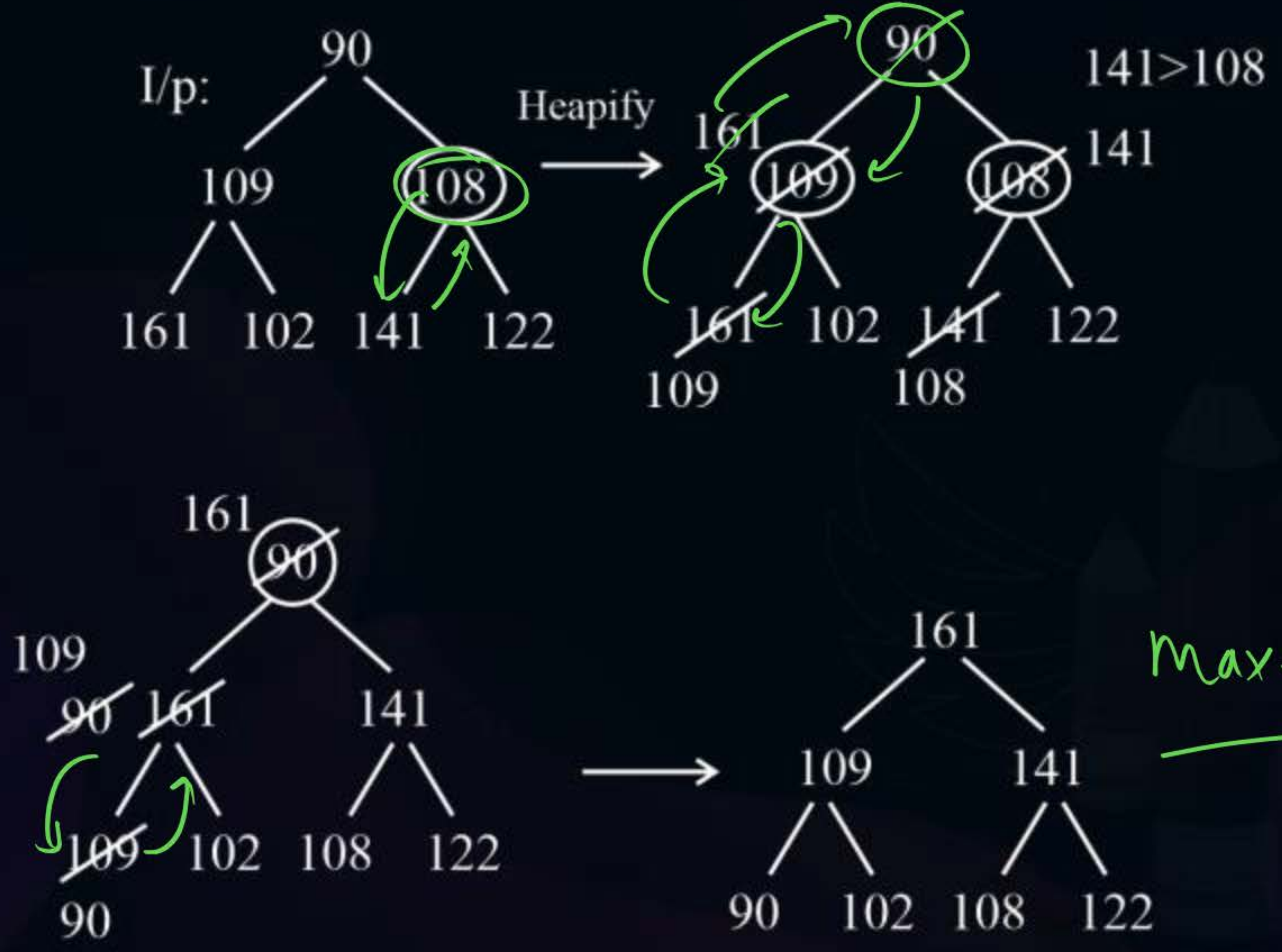
Important:

- Check for $\boxed{\text{adjustments}}$ from the 2nd last level ≈ (n/2 nodes at ~~last level~~ leaf)

Topic : Heap Algorithms

$108 < 141$

$109 < 161$



$90 < 161$

$90 < 109$

Max-Heap ✓

Important:

For i/p : A[n]

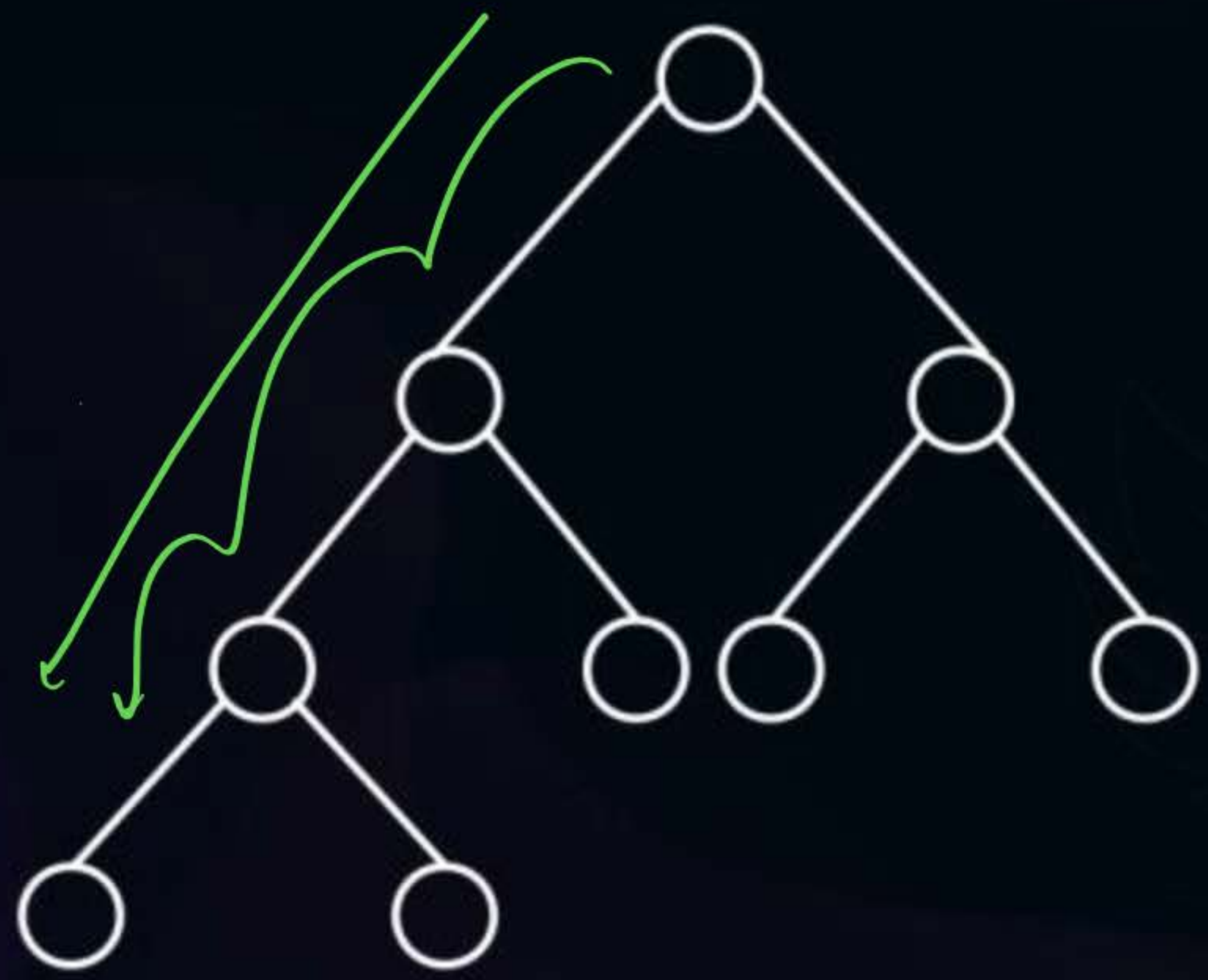
Heapify → TC : O(n) for Heap Creation

Topic : Heap Algorithm

Adjust Method:- → O(log n)

Heapify

Top-down



2nd Last level

Topic : Heap Algorithm

E.g.1. A: [100, 119, 118, 171, 112, 151, 132]

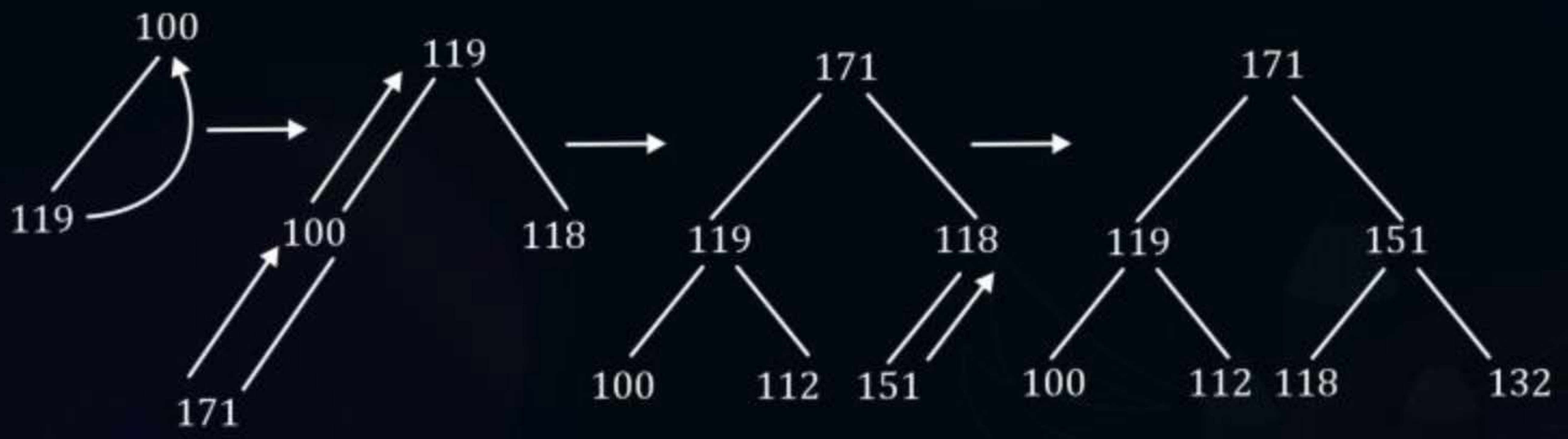
swaps ?

1) Insertion mtd. ?

2) Heapify ?

Topic : Heap Algorithm

1. Insertion method:-



Total Swaps:- 1 + 2 + 1 = 4

2. Heapify / Build Heap:

[100, 119, 118, 171, 112, 151, 132]

Max-Heap

118 < 151

119 < 171

100 < 171

100 < 119

4 Swaps

Topic : Heap Algorithm

Idea


```
procedure ADJUST(A, i, n)
integer i, j, n;
j = 2 * i; item ← A(i)
while i ≤ n do
        if (j ≤ n A (j) < A(j + 1)) then
        j ← j + 1 //j points to the larger child//
end if
if (item ≥ A(j)) then
        exit // a position for item is found //
else
        A(⌊j/2⌋) ← A(j)// move the larger child up a level//
j ← 2* j
end if
repeat A(⌊j/2⌋) ← item
end ADJUST
```

Topic : Heap Algorithm

procedure HEAPIFY (A, n)

// Readjust the elements in A(1 : n) to form a heap//

Integer n, I

for i = $\lfloor n/2 \rfloor$ to 1 by – 1 do

    call ADJUST (A, i , n)

repeat

end HEAPIFY

$n/2 \rightarrow 1$

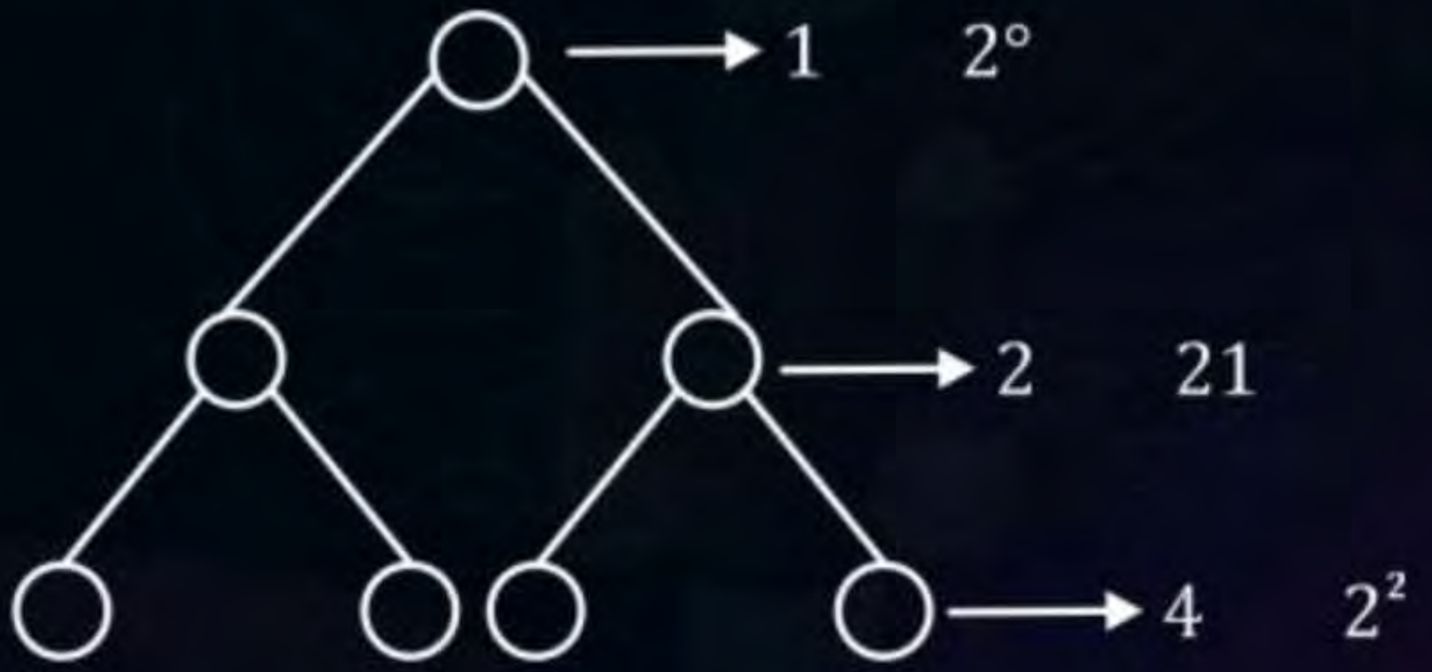
Topic : Heap Algorithm

Time complexity of Build-Heap (Heapify)

(logical approach) → (worst case)

- Max number of nodes at nay level I = $2^{(i-1)}$ (root is at level 1)
- Max number of level comparisons / moves for a nodes present at any level I that is getting adjusted.

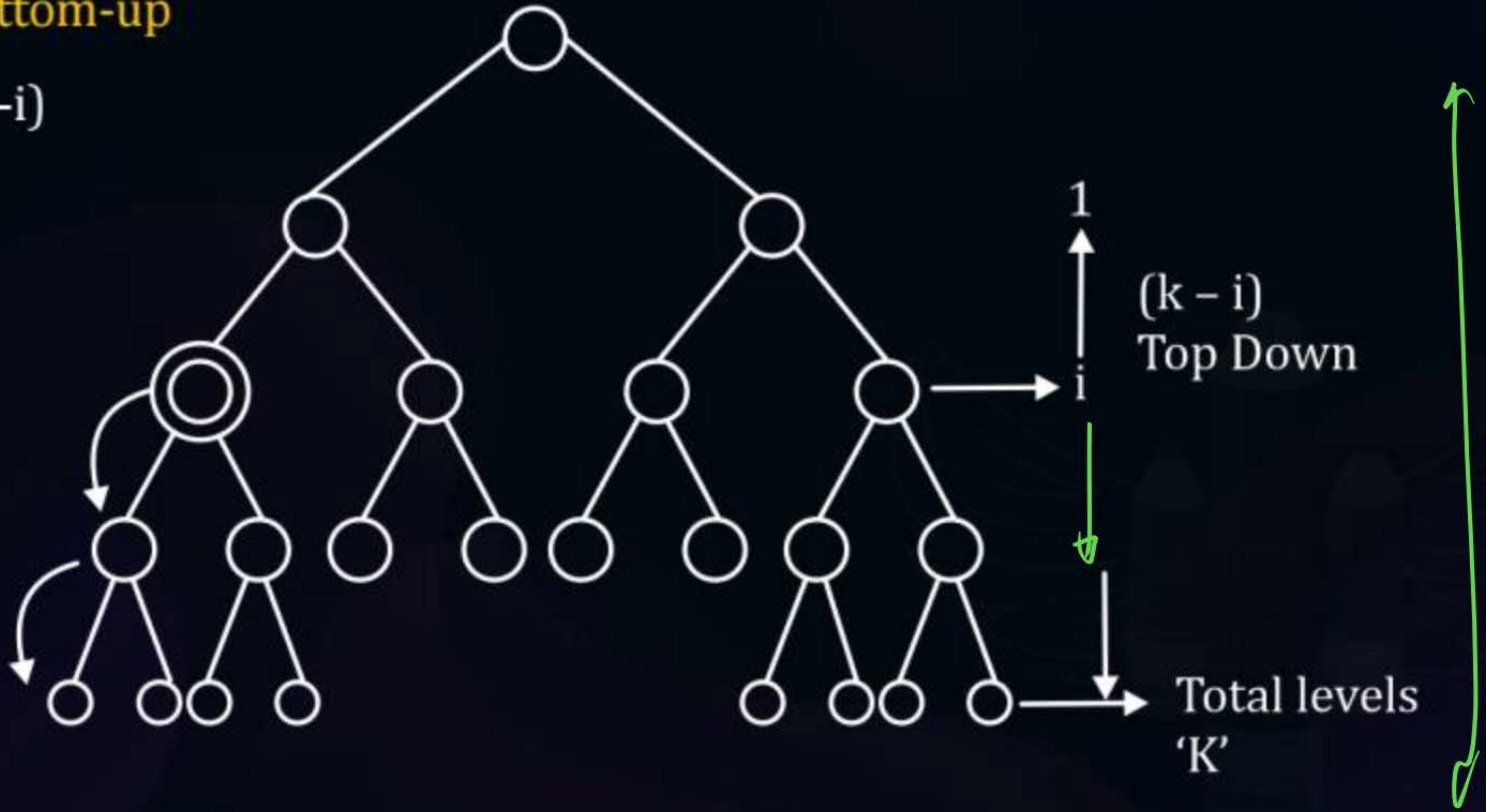
= (k –i)



Topic : Heap Algorithm

(i –1) Bottom-up

(k –i)



Topic : Heap Algorithm

3. For all the nodes at any level I, the total number of comparisons = (k –i) * $2^{(i-1)}$

4. Total number of comparisons in worst case for all the nodes combine~~s~~d (level 1→ k)

$$T(n) = \sum_{i=1}^{k} (k-i) * 2^{(i-1)}$$

$$= \sum_{i=1}^{k} (k-i) * \frac{2^i}{2}$$

$$= \frac{1}{2}\left[\sum_{i=1}^{k} (k-i) \times 2^i\right]$$

$$= \frac{1}{2}\left[\sum_{i=1}^{k} k * 2^i - \sum_{i=1}^{k} i \times 2^i\right]$$

Topic : Heap Algorithm

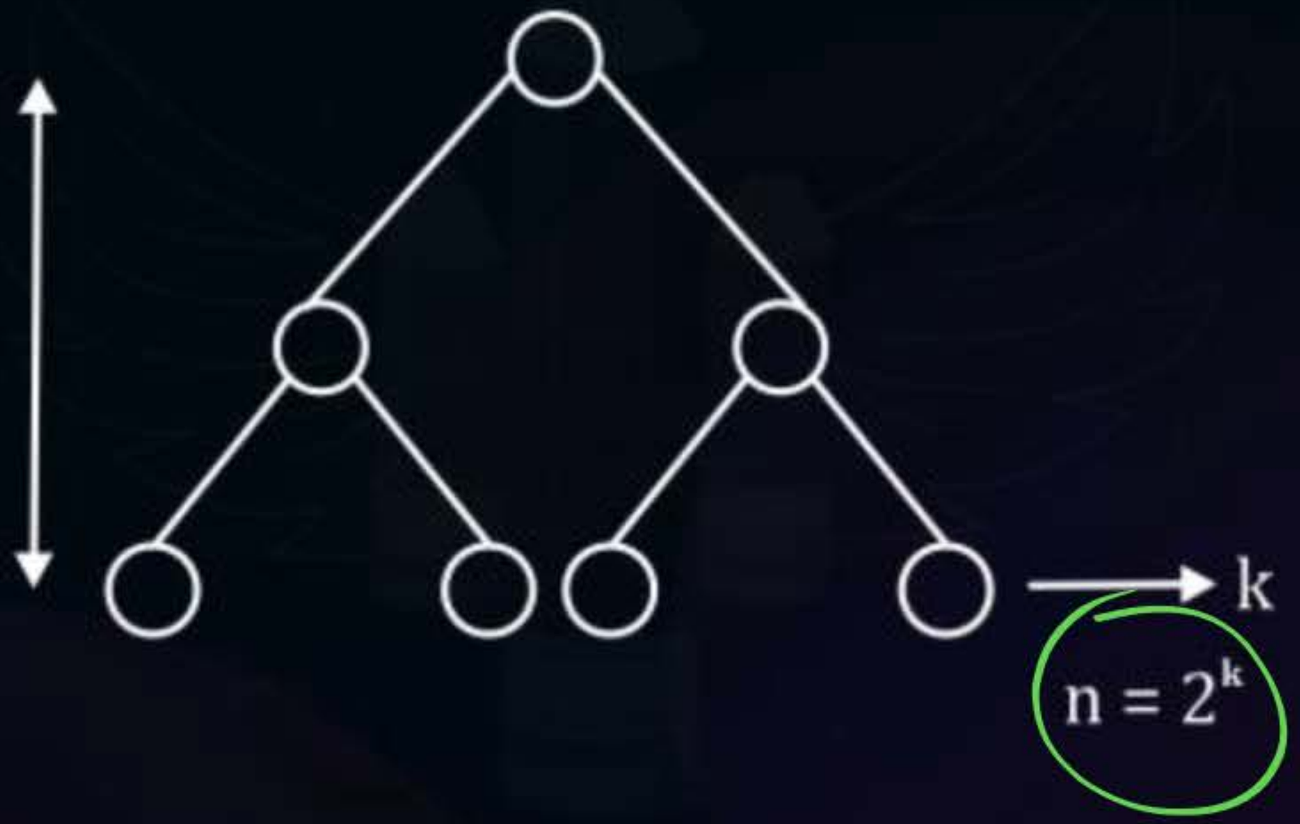
$$=\frac{1}{2}\left[\underline{k(2^{k+1}-2)}-\left(\underline{(k-1)*2^{k+1}+2}\right)\right]$$

$$=\frac{1}{2}[2\times k\times 2^{k}-2\times k-2-k\times 2^{k}\times 2+2^{k}\times 2]$$

$$=\boxed{2^{k}-k-1} \Rightarrow \underline{O(n)} \checkmark$$

$k = \log_2 n$

$K = \log_2 n$



Topic : Heap Algorithm

$$T(n) = 2^k - k - 1$$

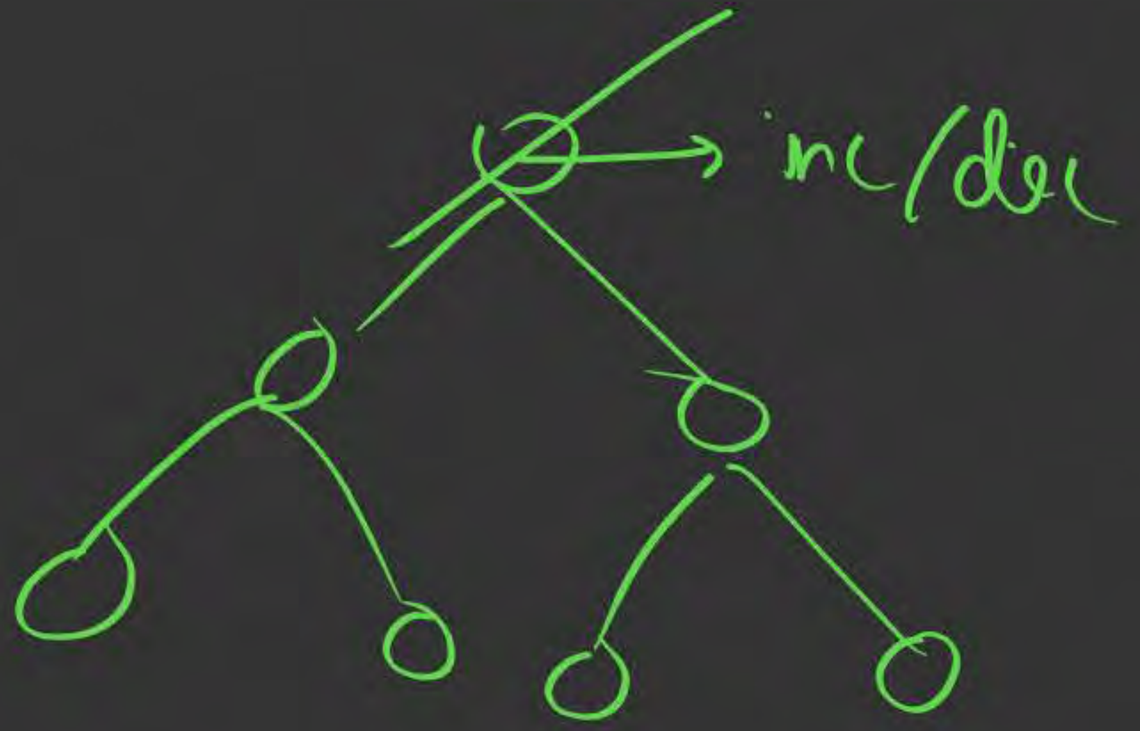
$$= n - \log_2 n - 11$$

$$T(n) = O(n)$$

To construct heap with n elements.

$$\boxed{\begin{array}{l}\text{Heapify} \rightarrow O(n)\\ \text{Insertion} \rightarrow O(n\log n)\end{array}}$$

~~CBT → readjust adding a node one by-one to a Heap.~~


inc/dec

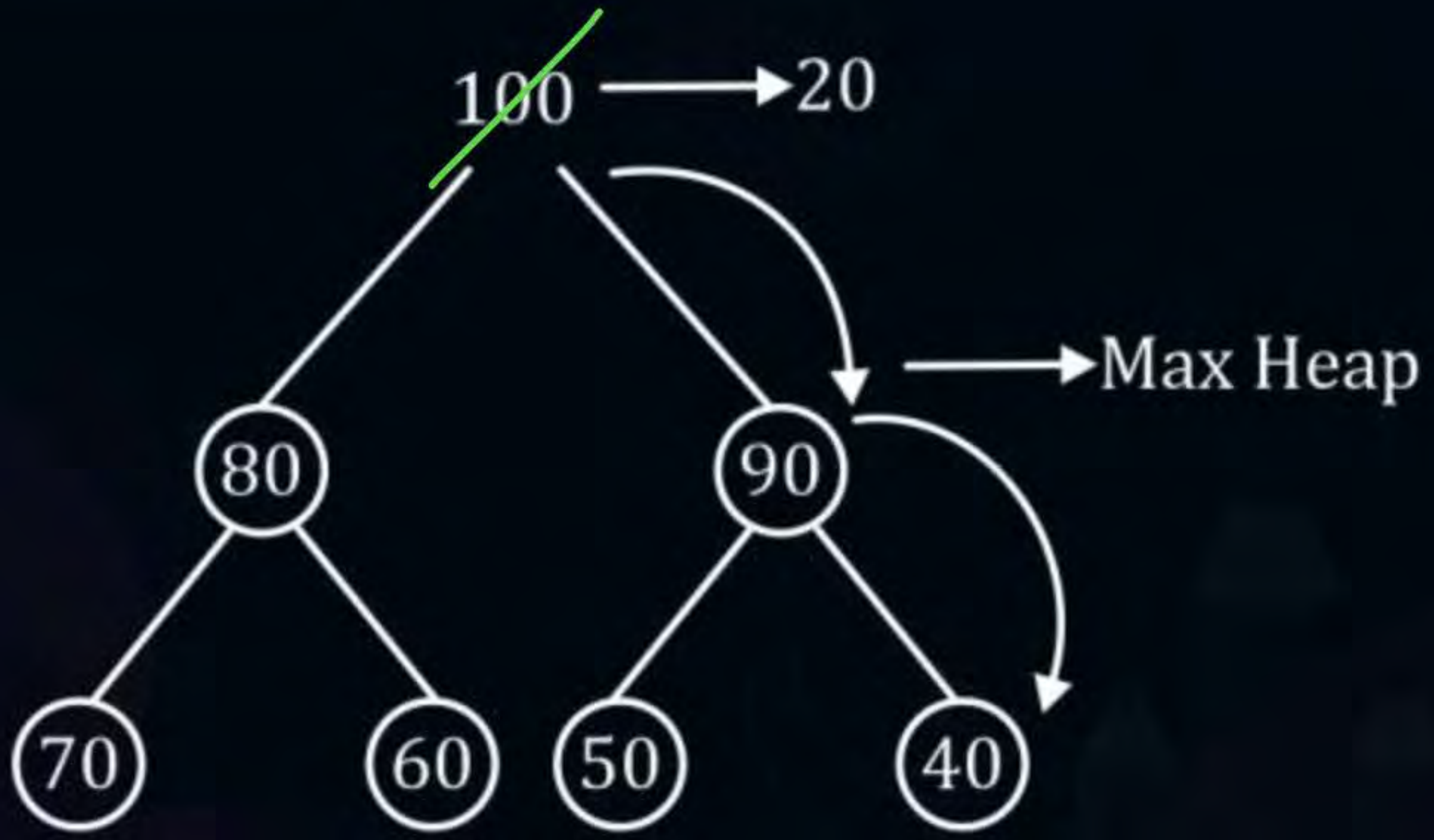
Topic : Heap Algorithm

Increase /Decrease key operation: TC

Given a heap with 'n' elements the time complexity to perform the ~~increasing~~ increase key/decrease key operation is______. $\boxed{\cancel{O(\text{long})}}$

$O(\log(n))$

Topic : Heap Algorithm



After decrease/ increase operation, we have to readjust to make it a valid heap again.

$= O(\log n)$

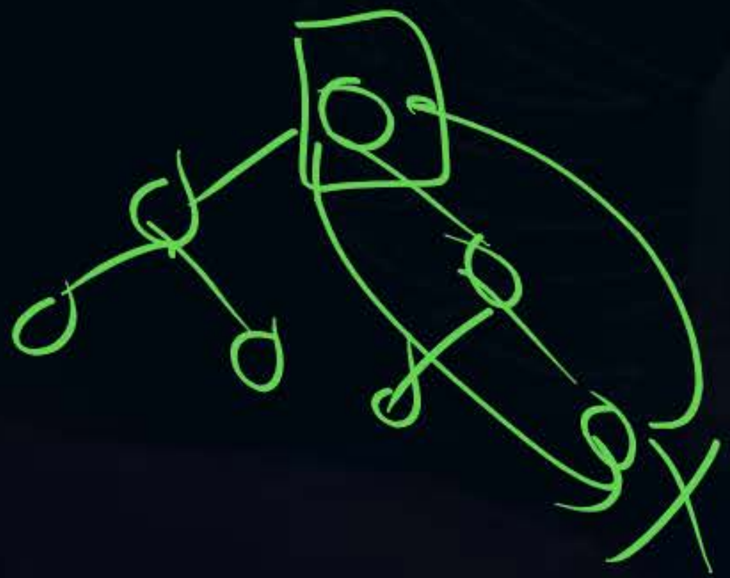
Topic : Heap Algorithm

Delete operation in a Heap

For a max Heap → Max elements
Min- Heap → min elements

The root node is getting deleted.

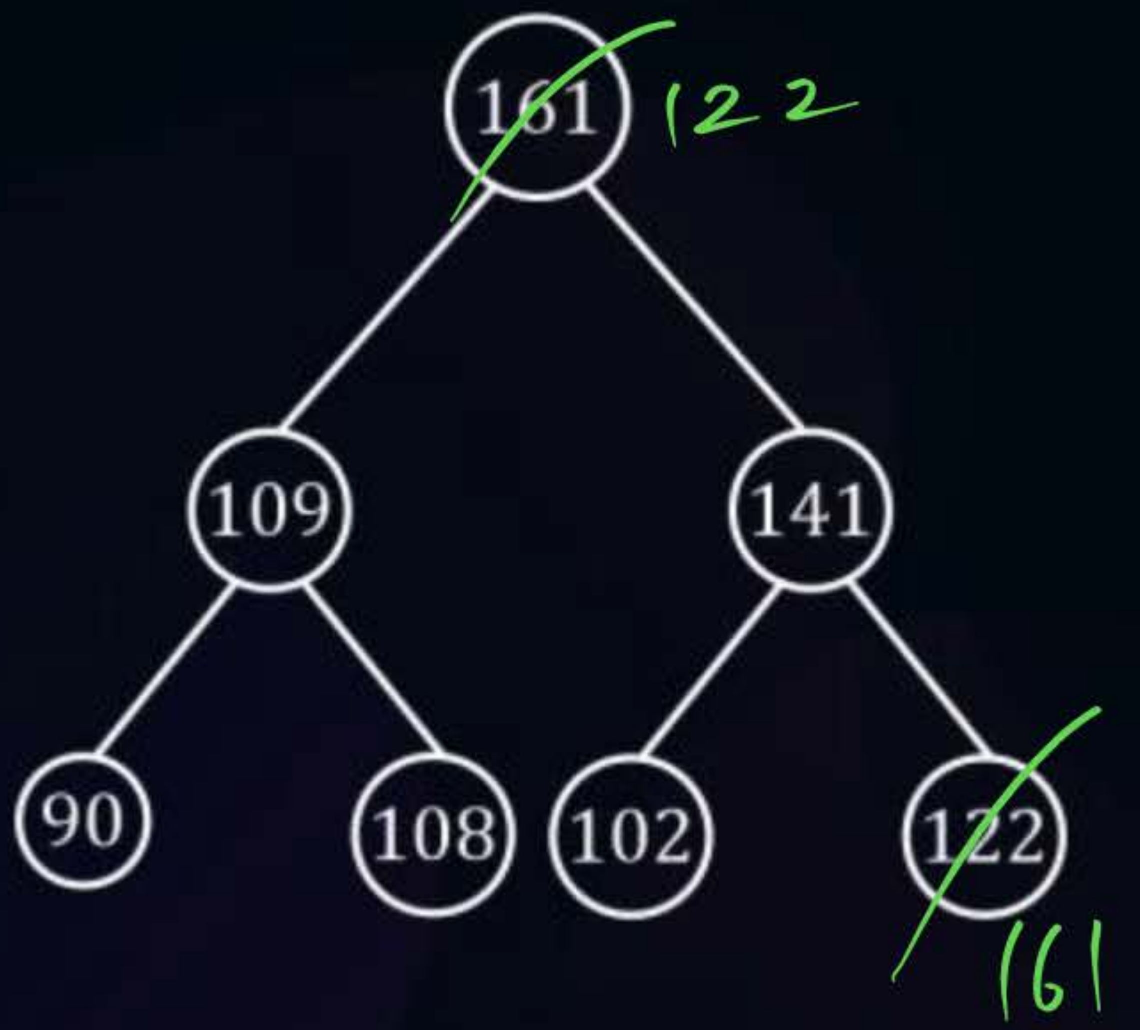
1. Swap Root with the last elements swap (A [1] , A [n])
2. Remove A [n]
3. Adjust A [1]



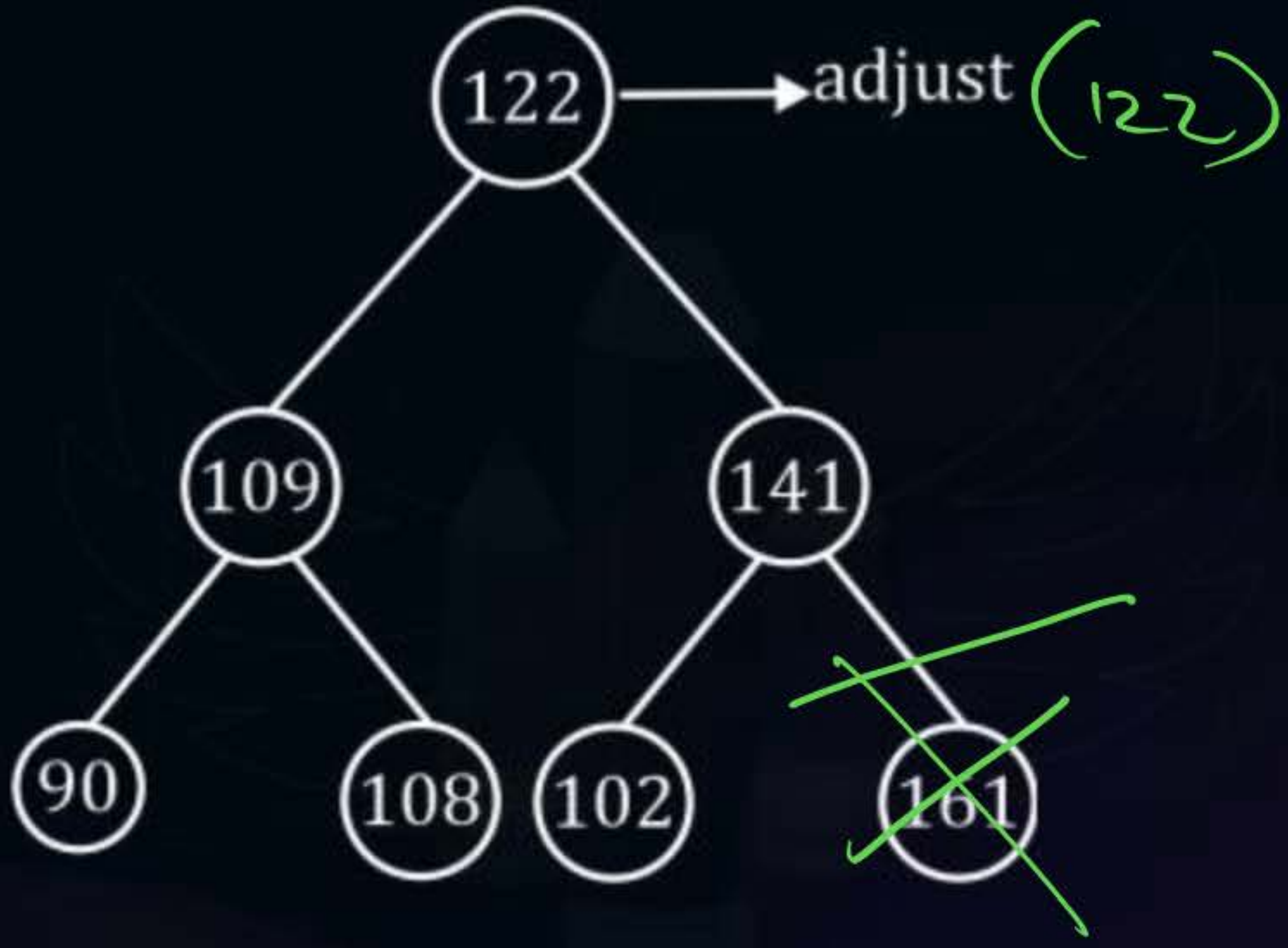
Topic : Heap Algorithm

E.g.2. Given Heap:

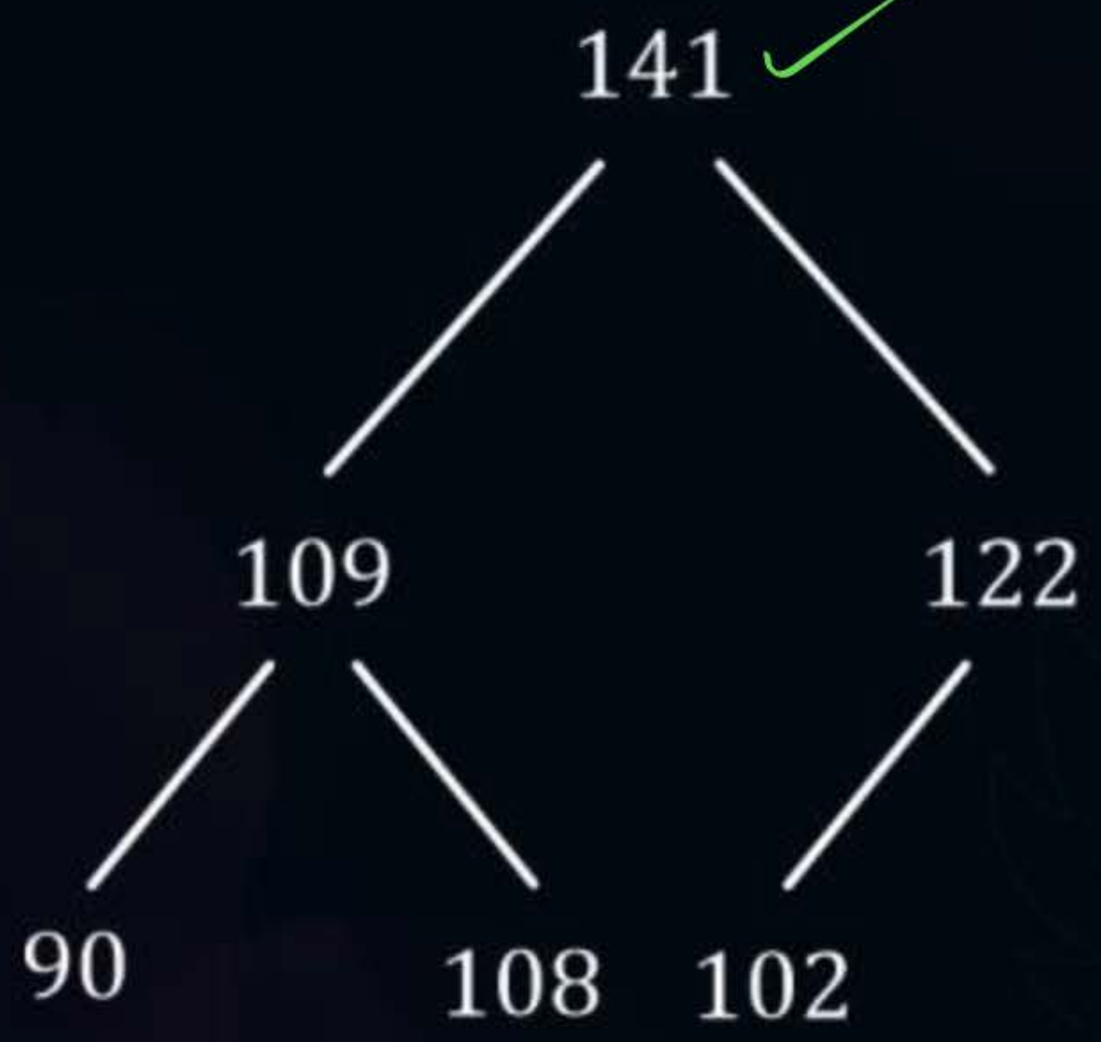
del (161)



Step-1: Swap ~~(16, 1, 12, 2)~~ 161, 122



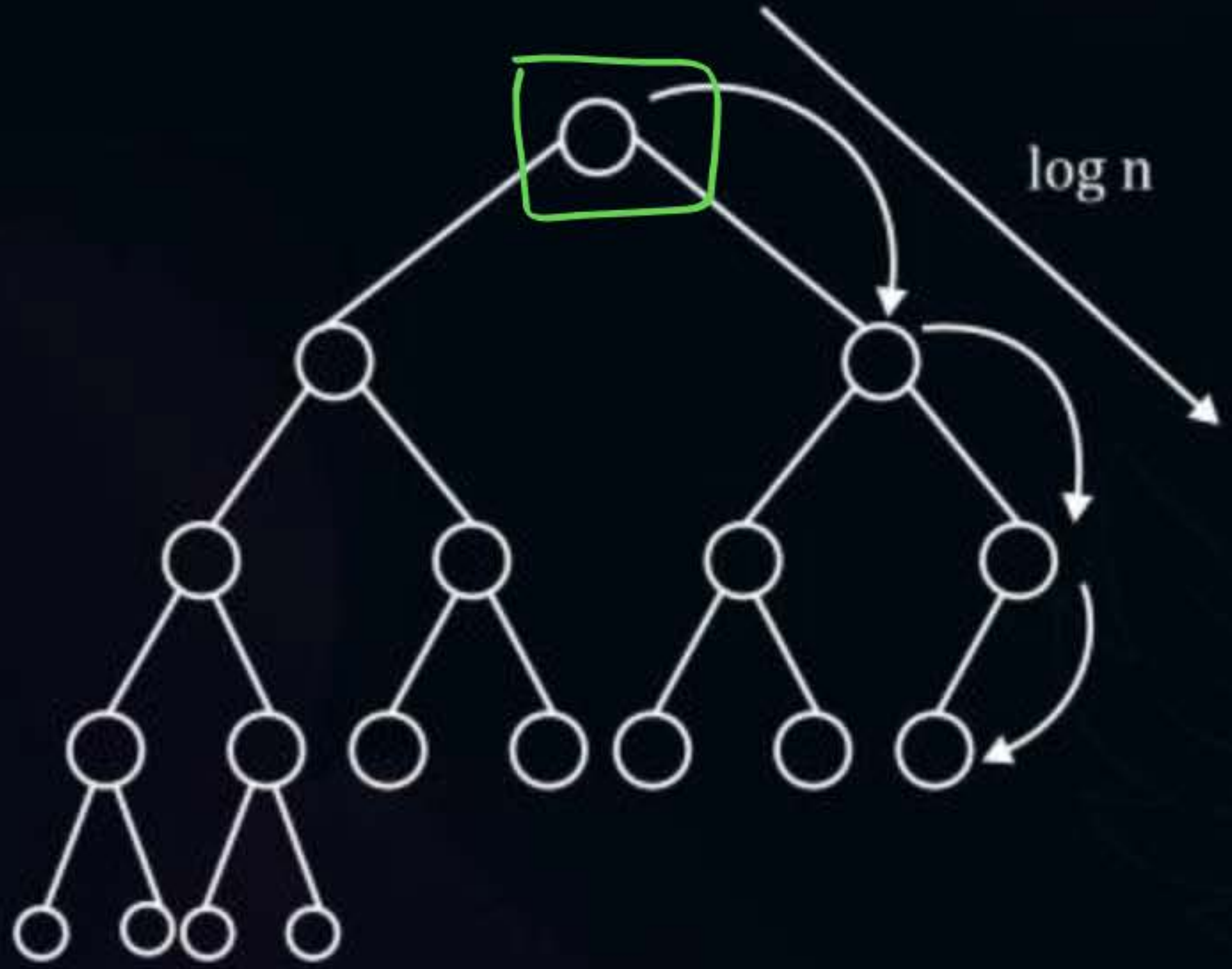
Topic : Heap Algorithm



→ Final Heap after deleting 161.

Topic : Heap Algorithm

Time complexity of a Delete Operation :



Delete → O(log n)

Ater removing you have to adjust the new root to make sure its back a valid Heap.

Topic : Heap Algorithm

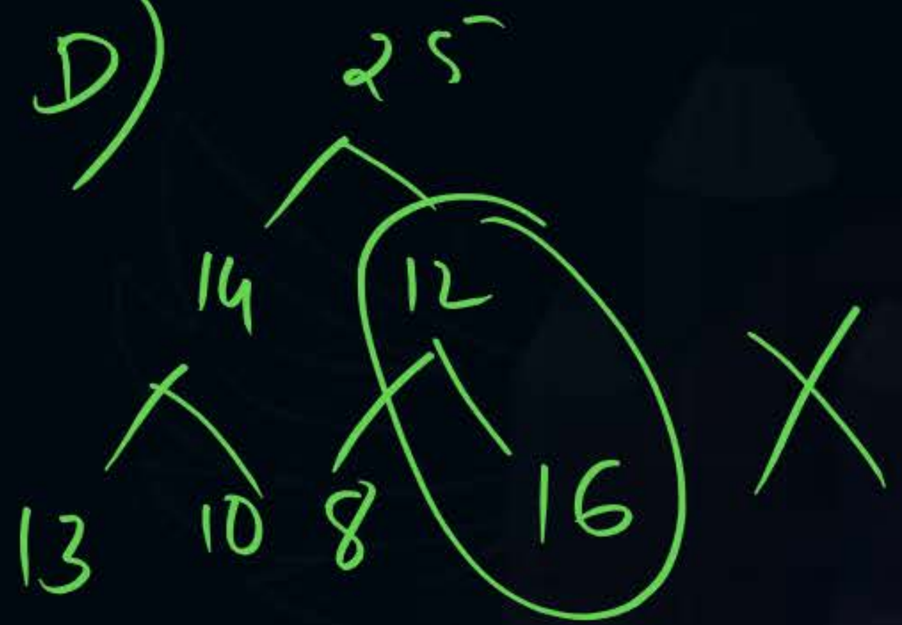
95%

90.1%

#Q. Which Array Representation is a valid Binary Max-Heap

(7, 2, 10, 4)

D)



A <25, 12, 16, 13, 10, 8, 14>

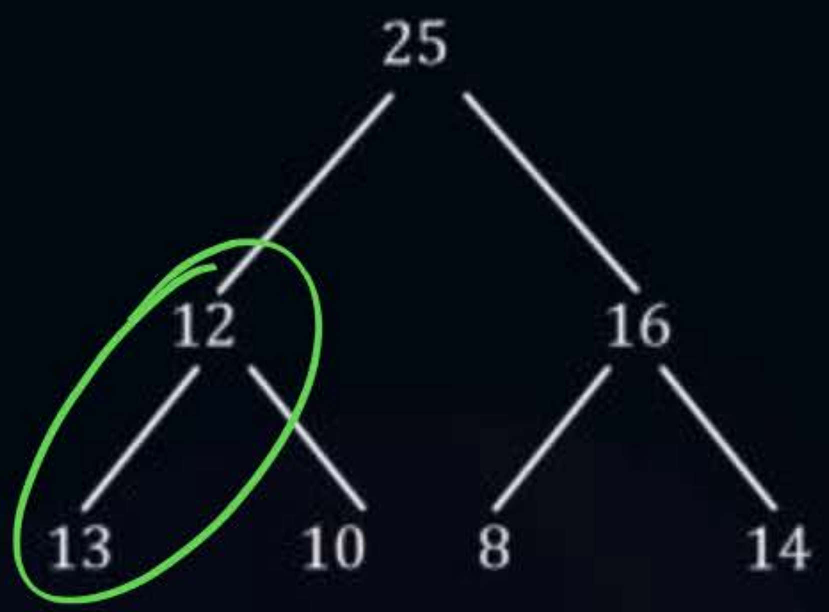
B <25, 14, 16, 13, 10, 8, 12 > ✓

C <25, 14, 13, 16, 10, 8, 12>

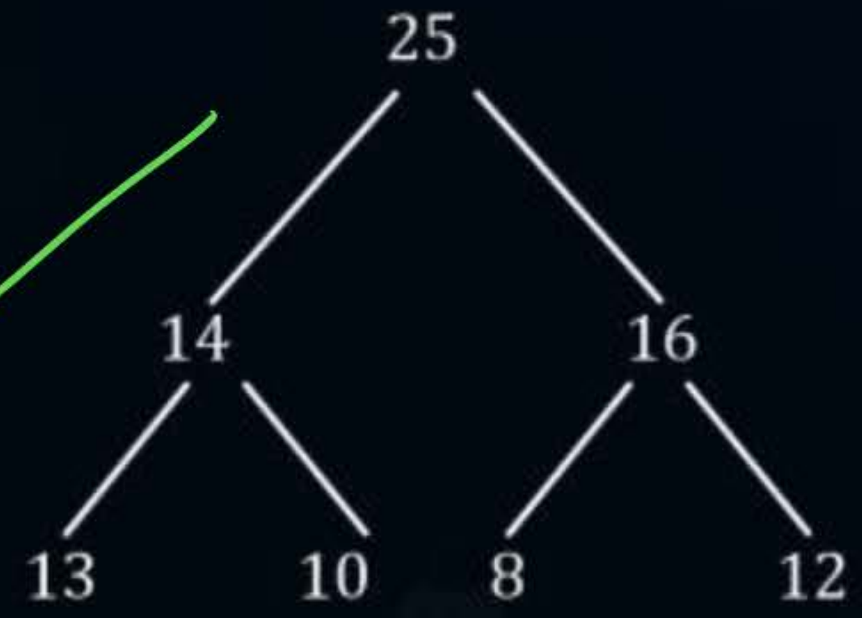
D <25, 14, 12, 13, 10, 8, 16>

Topic : Heap Algorithm

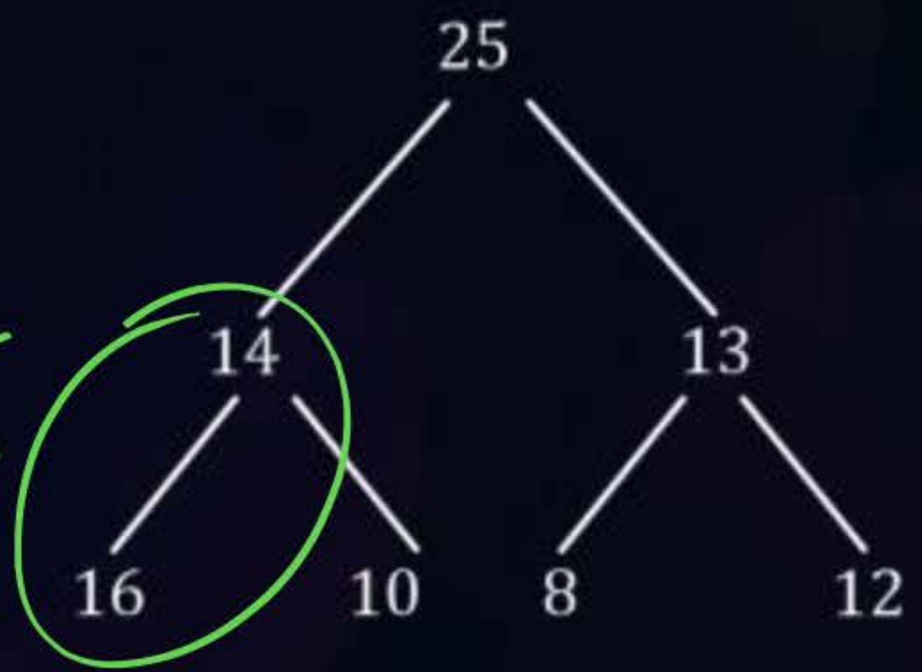
(A)



(B)

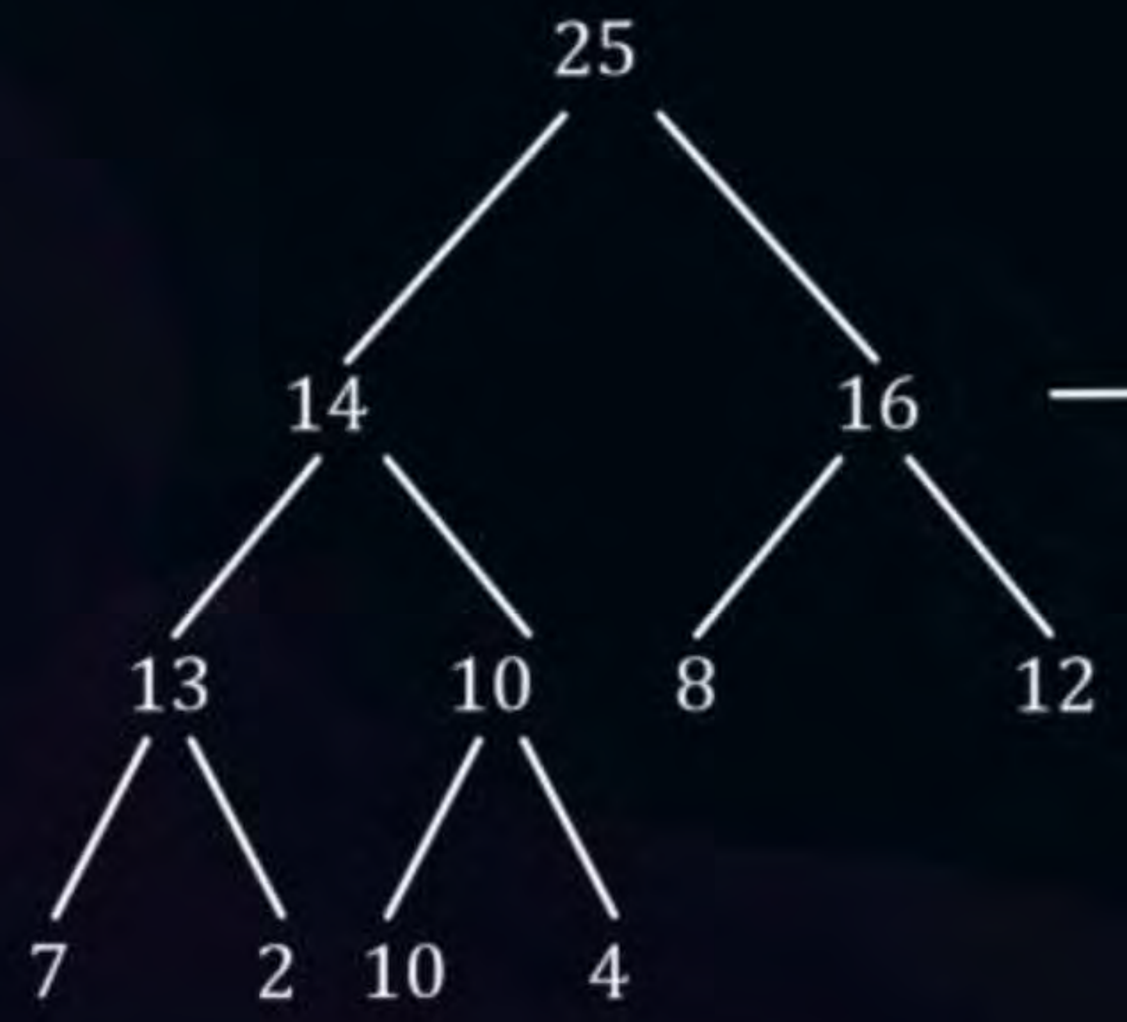


(C)



Topic : Heap Algorithm

#Q. To the valid Heap of Previous Question insert elements <7 2 10 4>. Indicate the resultant Heap in Array.
How many total swaps after insertion?



→ 0 Swaps

Topic : Heap Algorithm

#Q. Which one is ~~vale3-are~~ maximum Heap Array representation 72.2%

valid ternary

A < 1, 3, 5, 6, 8, 9 >

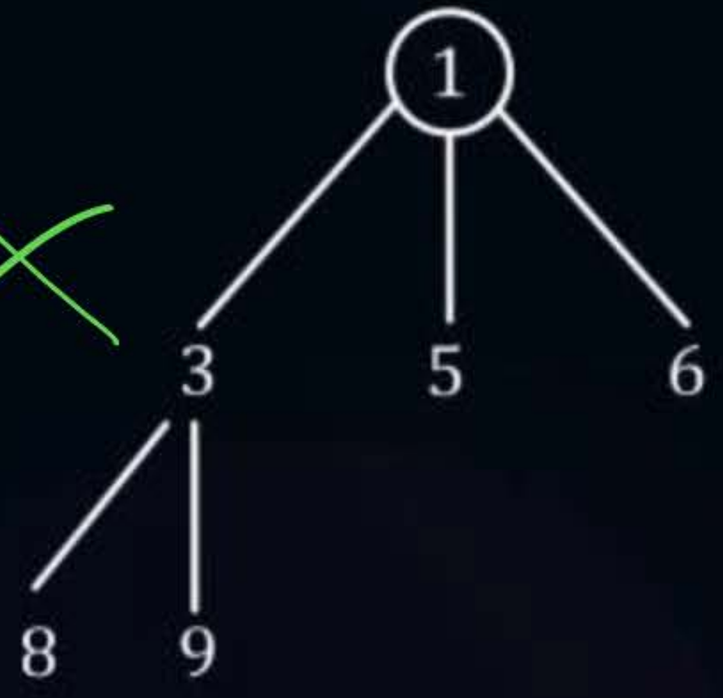
B < 9, 6, 3, 1, 8, 5>

C <9, 3, 6, 8, 5, 1>

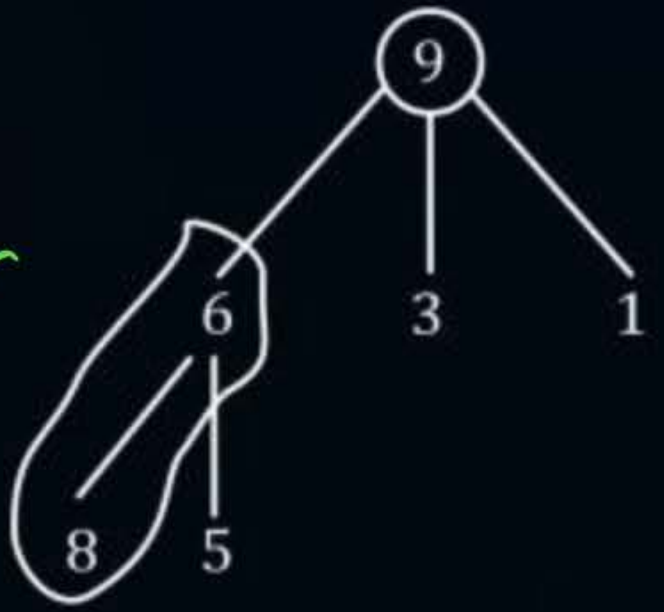
D <9, 5, 6, 8, 3, 1>

Topic : Heap Algorithm

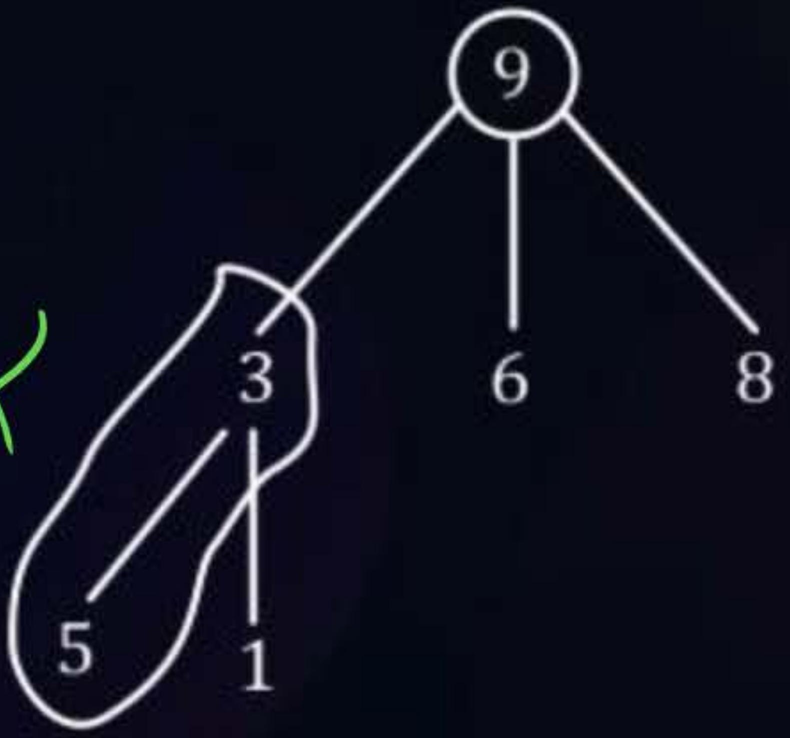
(A)



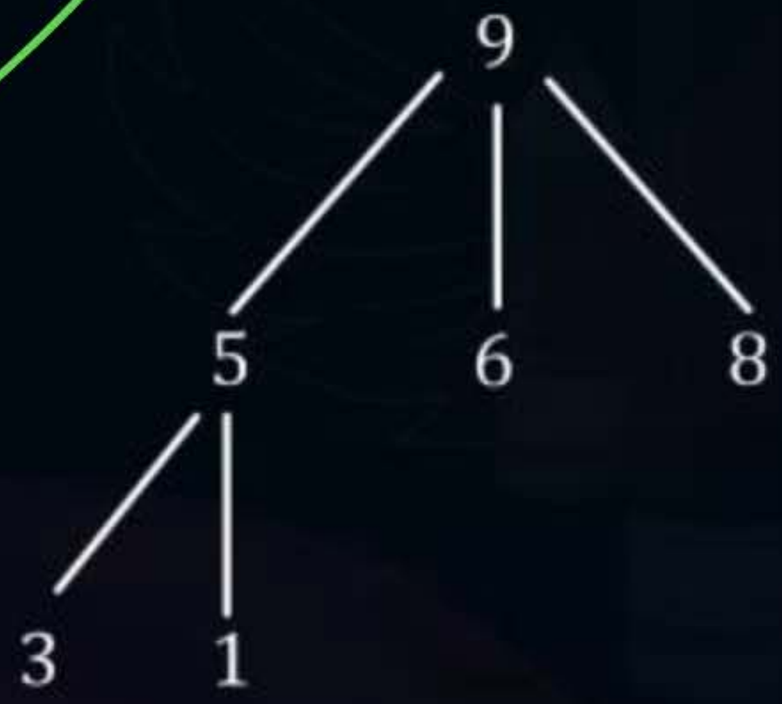
(B)



(C)



(D)



Heap Sort

Topic : Heap Algorithm

procedure HEAPSORT (A, n)

// A(1 : n) contains n elements to be sorted.//

1. call HEAPIFY (A, n)

2. for i ← n to 2 by – 1 do

$n \rightarrow 2$

    t ← A(i); A(i) ← A(1); A(i) ← t → swap(A(i), A(1))

    call ADJUST, (A, 1, i – 1)

    ✯ $(1 \rightarrow i-1)$

  repeat

end HEAPSORT

Topic : Heap Algorithm

Explanation:-  CBT → Heap

Idea

root→ always max element in the existing Heap.

Logic:-

Every time a root elements is deleted (max element) ~~to~~ the delete operation will re-adjust the such that the new maximum (max among remanning elements) becomes the root node (keep repeating this process.)

Topic : Heap Algorithm

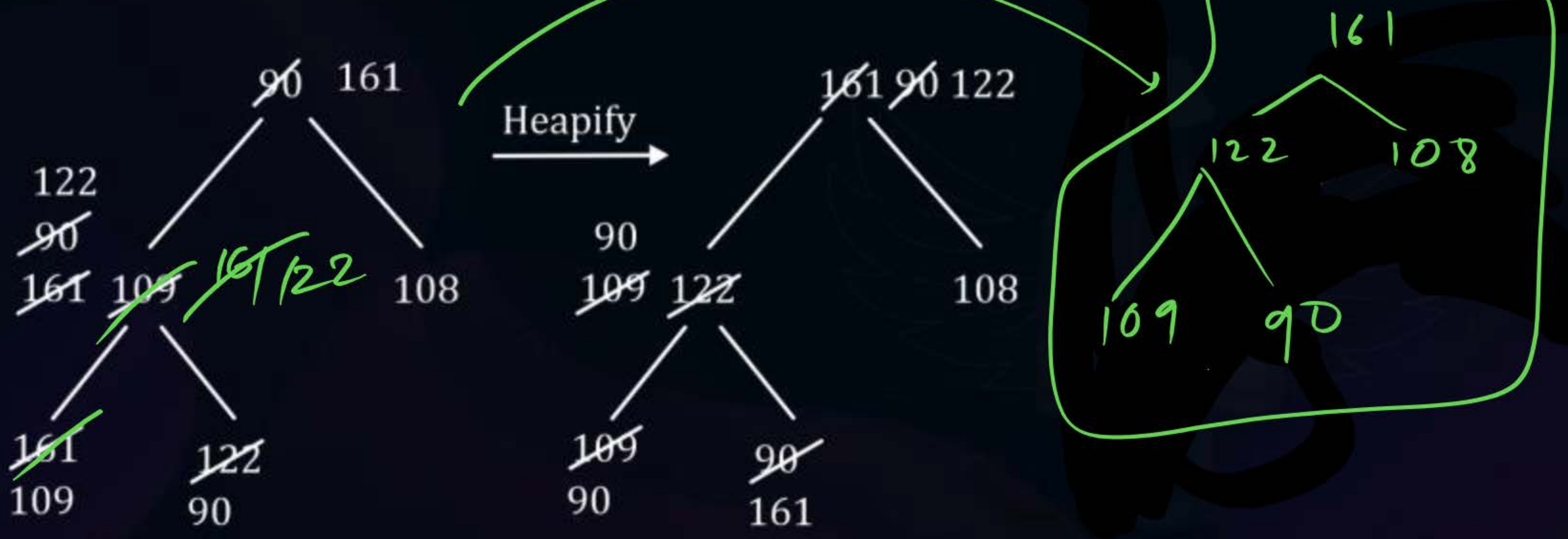
Dry Run ~~equation:~~

Sort given array in ascending

A :				
90	109	108	161	122

CBT

max Heap



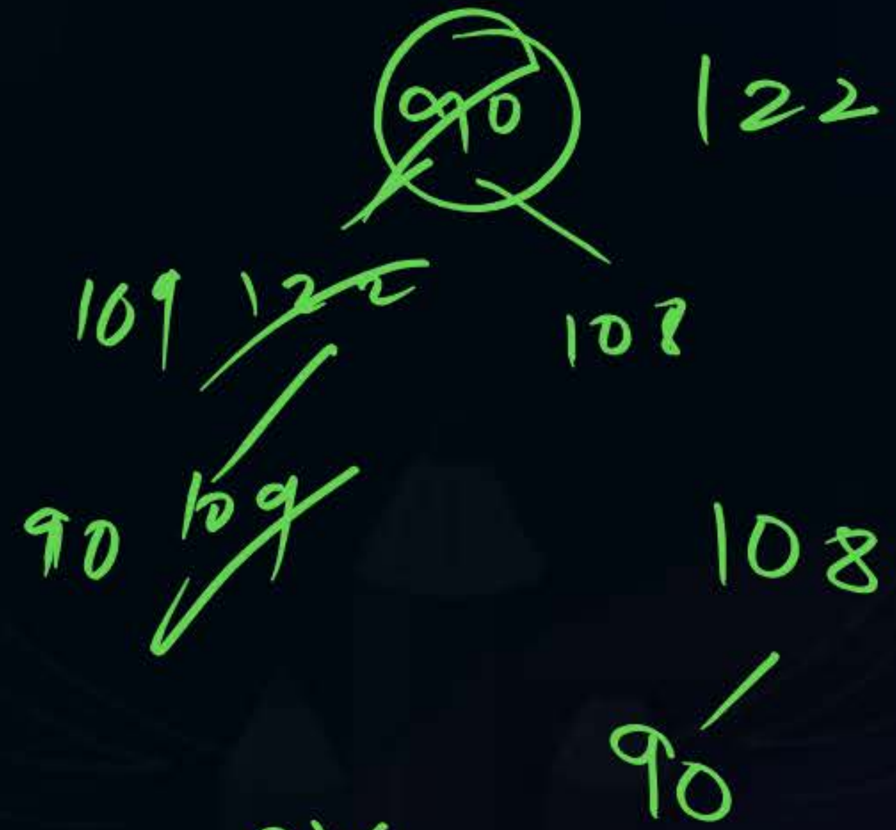
Topic : Heap Algorithm

Step by step iteration

given:

Heapify

1	2	3	4	5
90	109	108	161	122
161	122	108	109	90
90	122	108	109	161
122	109	108	90	161
109	90	108	122	161
108	90	109	122	161
90	108	109	122	161



→ Sorted in ~~are~~ asc order

~~Output: [161, 122, 109, 108, 90]~~

~~Decreasing orders~~

Topic : Heap Algorithm

Time complexity analysis:

Time complexity in every case $= \theta(n + n \log n)$

$= \theta(n + n \log n) \longrightarrow \theta(n \log n)$

Space complexity $= \boxed{O(1) \rightarrow \text{Inplace}}$

Not stable

10 10 10

Topic : Heap Algorithm

HW

#Q. In a Binary Max-Heap with n elements, the smallest element can be found in time of _____.

WC by best Algo

A. O(nlogn)

B. O(n)

C. O(n^2)

D. O(1)

2 mins Summary

Topic

Topic

Topic

Topic

Topic

Heap Algos

THANK - YOU